

Beyond Fault Localization: A Trajectory-Level Study of LLM Agents for Microservice Root Cause Analysis

Qisheng Lu*, Aoyang Fang*, Junjielong Xu*, Jin'ao Shang[†], Songhan Zhang*,
Yifan Yang*, Xiaochuan Yan*, and Pinjia He*

*The Chinese University of Hong Kong, Shenzhen, China

{qishengl, aoyangfang, songhanzhang, yifanyang6, xiaochuanyan}@link.cuhk.edu.cn

siyuexi@foxmail.com, hepinjia@cuhk.edu.cn

[†]Xi'an Jiaotong University, Xi'an, China

jinao_s@stu.xjtu.edu.cn

Abstract—Existing evaluations of automated root cause analysis (RCA) for microservices uniformly assess diagnostic performance by endpoint correctness: whether a method localizes the responsible service. Although this criterion enables direct comparison, it provides no indication of the evidentiary basis for a diagnosis or the propagation route linking the fault source to the observed symptoms. Both are necessary for an on-call site reliability engineer (SRE) to assess whether an automated diagnosis warrants action. We therefore treat RCA as an observable diagnostic process. Our trajectory-level framework evaluates agent executions against manually curated service-level fault-propagation paths. Applied to a public microservice RCA benchmark, the framework supports an analysis of 3,500 diagnostic trajectories, characterizing where agents investigate and how they incorporate retrieved telemetry.

Our results reveal a disconnect between answer correctness and diagnostic quality. An agent may correctly localize the fault source yet fail to reconstruct its propagation across the affected services; recovering this path is harder and more discriminative than identifying the affected services alone. Successful investigations tend to remain on the fault-impact surface, act on retrieved evidence, and exhibit a broader query repertoire as the search reaches deeper. Unsuccessful investigations instead depart from the relevant route, fail to use decisive observations, or stagnate in shallow, repetitive queries. Erroneous diagnoses reduce to three recurring evidence-handling failures: decisive evidence is omitted, retrieved evidence is misinterpreted, or unsupported inference substitutes for missing evidence.

We operationalize this failure taxonomy as DIAGGUARD, a two-stage defense-in-depth architecture: *grounding* systematically surveys the available observations before localization, and *verification* audits the diagnosis against them. In an independent validation setting with a different model, benchmark, and service topology, the prototype raises Acc@1 from 43.5% to 52.5%. These findings indicate that trajectory-level evaluation can expose limitations hidden by final-answer metrics and that recurring failure patterns provide an actionable basis for improving automated RCA.

Index Terms—Root Cause Analysis, Microservices, LLM Agents, AIOps

I. INTRODUCTION

Microservice architectures underpin large-scale cloud applications [1], where local faults can propagate across loosely

Pinjia He and Aoyang Fang are co-corresponding authors.

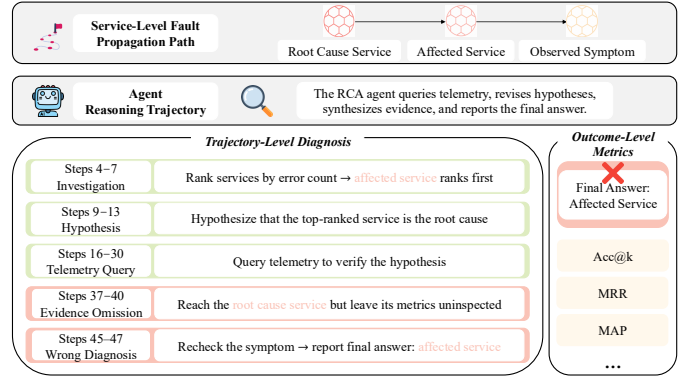


Fig. 1: Outcome-level evaluation captures only an RCA diagnosis’s final answer and scores; the diagnostic trajectory explains the wrong diagnosis: the agent reaches the fault source but omits decisive evidence before reporting the final answer.

coupled services and trigger cascading failures. When such incidents occur, on-call site reliability engineers (SREs) must diagnose the fault source from distributed traces, logs, and metrics before remediation can begin [2]. Because remediation acts on a diagnosis, the SRE needs not only a root-cause label but also the evidence and reasoning that make it trustworthy.

Automated root cause analysis (RCA) has produced a large body of methods [3]–[5]. Benchmark-driven studies usually turn RCA into a service-localization task: predict which service caused the incident. Top- k localization accuracy (Acc@ k), mean reciprocal rank, and mean average precision make this task comparable across methods, but they collapse the diagnostic process into a final label [6]. That collapse matters because the same correct label can be supported by strong or weak evidence, and a wrong label is only useful for continuous improvement if its failed reasoning can be inspected (Fig. 1). Across incidents, inspected reasoning failures can become reusable failure modes and guide better automated RCA design.

Tool-using LLM systems make this missing diagnostic

process newly visible. General-purpose coding and deep-research agents now plan over many steps, invoke tools, and synthesize evidence [7], [8]. LLM-based RCA methods follow the same pattern by querying telemetry and revising hypotheses during diagnosis [9], [11], [12], [21]. Because these agents act through observable steps, their runs leave trajectories: records of agent messages, telemetry queries, and tool-returned evidence. Such trajectories can expose how an agent investigates an incident, not only what answer it returns.

However, raw trajectories cannot be evaluated directly as process-level evidence. Prior RCA work studies causal graphs and fault propagation for localization [3]–[5], and agent studies analyze tool traces [7], [8]. However, causal graphs in prior work capture statistically inferred service dependencies, not the incident-specific route a fault actually traveled; agent-trace analyses, in turn, lack a domain-level ground truth to evaluate against. To evaluate diagnostic trajectories in RCA, what is still missing is a reference for what the diagnostic process should have recovered. For our setting, that reference is a manually validated service-level fault propagation path: the route by which a fault moves from the faulty service to the observed symptom through service-dependency hops. RCABench [13], the microservice benchmark used for our main study, provides final root-cause labels but not these process-level annotations. Even with such paths, trajectories still need normalization because heterogeneous agents expose different trace formats and tool interfaces. Without both process-level ground truth and normalized steps, existing metrics cannot say whether an agent navigated relevant evidence, what operational purpose each step served, or why wrong diagnoses occurred.

We address this gap with a trajectory-level characterization framework built around service-level fault propagation paths. These paths define the fault-impact surface: the services and edges through which a fault reaches the observed symptoms. To compare agent behavior against this surface, we normalize each run into agent messages, telemetry queries, and tool-returned evidence. The framework applies outcome-level metrics to all six frameworks and adds step-level trajectory analysis where the shared telemetry interface makes tool traces comparable. The resulting analysis first checks whether a run finds the root service and reconstructs the propagation path. The analysis then explains the route the agent took by mapping how the agent navigates the fault-impact surface, labeling SRE diagnostic intent and coding the failure modes behind wrong diagnoses. Together, these steps make RCA trajectories measurable, comparable, and useful for mining recurring diagnostic failures.

With this framework in place, the study pursues two questions. The first is *characterization*: what separates a sound diagnostic process from a failing one? We pursue it at three progressively deeper levels (RQ1–RQ3), moving from the outcome and cost of localization, to how an agent navigates the fault-impact surface and what diagnostic intent drives each query, to why wrong diagnoses occur. We compare six frameworks under a shared backbone model and isolate the backbone-model effect by rerunning one framework on a second model. The trajectory-level analysis (RQ2–RQ3) focuses on

the four frameworks whose tool calls are SQL queries against a shared telemetry store, because only they expose comparable action traces. The second is *intervention* (RQ4–RQ5): can the characterized failures become a usable improvement? We distill them into DIAGGUARD, a defense-in-depth architecture that wraps the diagnostic agent in a grounding defense (the *Grounder*) and a verification defense (the *Verifier*). On a held-out setting, we test whether DIAGGUARD raises localization and isolate which defense drives the gain.

Our study yields three core findings. 1) Localization difficulty is uneven, accuracy falls as the causal chain deepens, and reconstructing how a fault propagates between services is harder than identifying which services it reaches, even when the fault source is correctly localized. 2) Success and failure hinge on the diagnostic process itself: effective runs stay on the fault-impact surface and act on the evidence they retrieve, and the breadth of their intent repertoire sets how deep that search reaches; failing runs drift off the surface, leave decisive evidence unused, or circle in shallow loops. 3) Wrong diagnoses collapse into a small, evidence-handling failure taxonomy, and experience distilled from these modes remains effective on a held-out backbone model, dataset, and service topology, carrying the characterization into intervention.

This paper makes the following contributions:

- **Annotated propagation-path dataset.** We manually annotate service-level fault propagation paths for a microservice RCA benchmark, supplying process-level ground truth beyond final root-cause labels.
- **Trajectory-level characterization framework.** We introduce a framework that normalizes heterogeneous RCA trajectories and makes them comparable against the annotated propagation paths.
- **Process-level empirical study.** We analyze 3,500 reasoning trajectories across seven framework–model configurations (six frameworks under a shared model, one rerun on a second model), linking diagnostic behavior and failure modes to final RCA outcomes.
- **Failure-mode-derived intervention.** We use the mined failure modes to design DIAGGUARD and validate it on a held-out backbone model, dataset, and service topology, showing that trajectory-level failure analysis can guide an intervention beyond the original characterization setting.

We will release the annotated propagation-path dataset, trajectory-analysis pipeline, the failure-mode codebook, and the DIAGGUARD defense prompts to support reproducible process-level research on automated RCA.

II. RELATED WORK

Our work sits at the intersection of microservice RCA methods, tool-using LLM agents, and agent trajectory analysis. We review traditional and LLM-based RCA (Sections II-A and II-B), the general-purpose agents we repurpose for RCA (Section II-C), and trajectory-analysis techniques (Section II-D). Across these lines, general-purpose deep-research and coding agents have not been systematically compared with RCA-specific designs on end-to-end RCA, and diagnostic processes

are either not exposed or not evaluated against process-level ground truth; our study addresses both gaps.

A. Traditional RCA in Microservice Systems

Automated RCA research spans four families of methods: statistical correlation [14], [15], graph-based fault propagation [16], [17], causal inference [10], [18]–[20], [22], and learning-based frameworks over multimodal observability data [23]–[25]. These methods return a root-cause ranking rather than a reasoning process; LLM-based and agent-based methods have recently emerged as a more process-oriented direction [5].

B. LLM-Based RCA for Microservices

The application of large language models to microservice RCA has produced two distinct lines of work, following the *workflow-versus-agent* distinction [26].

Workflow approaches. These methods preprocess multimodal telemetry into a structured prompt for single-pass inference. They target classification and confidence estimation [9], [27], [28], explanation and remediation [29]–[33], or enrich the prompt with structured knowledge such as mined code or offline causal graphs [21], [34]. A single snapshot of system state, however, cannot capture the progressive, hypothesis-driven nature of expert fault diagnosis.

Agentic approaches. These methods reason over multi-turn loops of diagnostic tool calls, telemetry queries, and hypothesis refinement. MABC [11] and OPENRCA [12] are representative iterative RCA agents, which we adopt as the two RCA-specific frameworks in our study (Section III-A); other designs add SOP-derived tools, self-consistency, recursive cross-alert memory, or coordinated SRE-role agents [35]–[39], and a cross-cutting line fine-tunes the backbone model specifically for RCA [40], [41], unlike the off-the-shelf agents our study examines.

Despite this progress, these methods are assessed almost exclusively at the outcome level, leaving the diagnostic process unexamined for lack of process-level ground truth.

C. Advanced Agents

General-purpose deep-research and coding agents now match or exceed domain-specific methods on knowledge-intensive tasks, raising the question of whether investigative architecture matters more than domain specialization. The DeepResearch Bench [8], [42] provides the first systematic evaluation of deep-research agents (DRAs); on the open-source side, AIQ [43], TASKWEAVER [44], and THINKDEPTH.AI [45] are representative frameworks, which we repurpose for RCA under the same harness (Section III-A). Commercial coding agents such as CLAUDECODE (Anthropic’s Claude Code) integrate with development environments to navigate codebases and iterate on solutions; we adapt one to RCA through the same harness, comparing it directly with RCA-specific agents.

D. Agent Trajectory Analysis

Inspecting agent trajectories has attracted growing interest: tools visualize and debug agent behavior [46], [47], and a parallel line mines historical trajectories to optimize agents [48]–[50].

For failure understanding, Bouzenia and Pradel [7] and Zhang et al. [51] analyze trajectories of software-engineering and multi-agent systems. Closest to ours, three concurrent efforts target RCA agents: Riddell et al. [52] study post-detection RCA, where externally extracted anomalies are analyzed by agents over a static system graph, Kim et al. [53] diagnose trajectories from a single OPENRCA architecture across multiple models, and STAR [54] repairs failing diagnostic stages post-hoc. We instead evaluate the end-to-end RCA capability of multiple real agent frameworks and backbone models directly over telemetry against manually annotated propagation-path ground truth and turn the mined failure modes into a preventive intervention (Section VI).

III. STUDY SETUP

This section establishes the study setup used throughout the paper: the evaluated agents, backbone models, and benchmarks. Fig. 2 gives the study design and the RQ1–RQ3 analysis pipeline; the validation (RQ4–RQ5) follows in Section VI-B.

A. Evaluated Agent Frameworks

We select six frameworks spanning two categories: general-purpose investigative agents and RCA-specific agents.

- THINKDEPTH.AI [45]: open-source deep-research framework, top-ranked before April 2026 [8], with a self-balancing ReAct-style loop [55].
- AIQ [43]: open-source deep-research framework, also top-ranked, with a requery-and-refine ReAct-style loop.
- TASKWEAVER [44]: Code-first plan-executor that decomposes a task into executable plans and tool-backed actions.
- CLAUDECODE: commercial terminal-style coding agent.
- OPENRCA [12]: RCA-specific agent driving an LLM-based loop over telemetry to produce root-cause predictions
- MABC [11]: RCA-specific two-stage multi-agent design in which agents analyze incident symptoms, then aggregate conclusions via a blockchain-inspired consensus step.

B. Backbone Models

Our study uses the following three backbone models (USD per 1M input/output tokens, as of 2026-02-15):

- qwen3.5-plus-2026-02-15: \$0.115/\$0.688.
- claude-sonnet-4.6: \$3/\$15.
- doubao-seed-2.0-pro-2026-02-15: \$0.44/\$2.22.

C. Benchmarks

We use two public benchmarks with different topologies and fault granularities.

- RCABench [13]: a microservice RCA benchmark with hierarchical service-to-code-level fault labels; we evaluate on a 500-case sample drawn by simple random sampling without replacement from its 1,430 cases.
- AIOps 2025 [56]: a microservices e-commerce RCA benchmark of 400 incidents over ten core services.

On RCABench, the empirical study (RQ1–RQ3) runs a horizontal comparison of the six frameworks under the shared Qwen and a vertical comparison of Qwen against Sonnet under

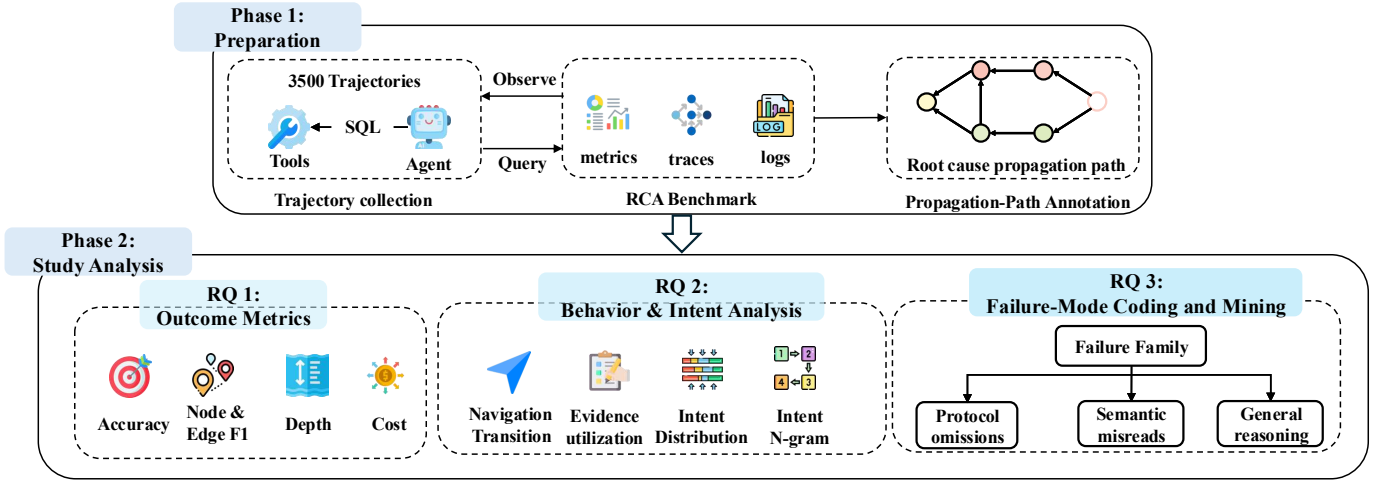


Fig. 2: **Study overview.** *Phase 1* collects 3,500 agent reasoning trajectories on RCABench: each agent issues SQL over telemetry, and we annotate every case’s fault propagation path. *Phase 2* characterizes them at three deepening levels: RQ1 outcome and cost (Section V-A), RQ2 behavior and intent (Section V-B), and RQ3 failure-mode families (Section V-C); and DIAGGUARD (RQ4–RQ5) appears in Fig. 7.

THINKDEPTH.AI to isolate the backbone-model effect, giving seven framework–model configurations (arms). The method evaluation (RQ4–RQ5) builds the baseline and DIAGGUARD on THINKDEPTH.AI and validates them on AIOps 2025 with the different-vendor Seed 2.0 Pro (the held-out setting).

IV. CHARACTERIZATION METHODOLOGY

This section details how we turn heterogeneous agent runs into path-grounded, comparable measurements: annotating ground-truth propagation paths and collecting trajectories (Section IV-A), defining outcome and difficulty metrics (Section IV-B), and analyzing trajectory behavior, intent, and failure modes (Section IV-C, Section IV-D).

A. Propagation-Path Annotation and Trajectory Collection

a) Propagation-path annotation.: RCABench labels each case’s root cause but not the service-level path to the SLO-alarming service, so we annotate this process-level ground truth ourselves. From the injection metadata (targeted service and fault window), the pre-/post-injection traces, metrics, and logs, and the static TrainTicket service-call graph [57], we reconstruct a service-level fault propagation graph $G^* = (V^*, E^*)$, the union of one or more propagation paths from the root cause to the symptom. Its nodes are the fault-affected services; each directed edge is a propagation step, realized through a logical service call or physical resource sharing and grounded in the abnormal spans, metric shifts, or log signatures within the fault window. This per-case G^* is the ground truth for Node F1 and Edge F1 (Section IV-B).

b) Trajectory collection.: We collect 3,500 reasoning trajectories. Each run yields thought-action-result triples; a final *compression prompt* then instructs the agent to reason over the collected evidence and report, in a uniform format, a predicted root-cause service and a self-reported causal propagation graph

$G = (V, E)$, whose edges need not be query-verified. Both ground the outcome metrics (Section IV-B).

B. Metrics Definitions

a) Trajectory sequence.: For each case, the agent’s reasoning is an ordered sequence of *rounds* $\mathcal{T} = [\tau_1, \dots, \tau_n]$, each a triple $\tau_i = (t_i, a_i, r_i)$ of thought, action, and result. For the four SQL-call frameworks, each action generates SQL over a shared telemetry interface; the two RCA-specific frameworks (MABC, OPENRCA) retain their native data access.

b) Reasoning budget.: We report each run’s per-case reasoning budget as three quantities: *rd*s (*effective rounds*: rounds issuing at least one tool call), *ktok* (mean input+output tokens, in thousands), and \$ (those tokens priced at the backbone model’s list price; Section III-B).

c) Root-cause localization.: Each case has a ground-truth root-cause set R_i^* of one or two services. Acc@1 scores a case correct when the agent’s predicted root cause $\hat{r}_i$ lies in R_i^* :

$$\text{Acc@1} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\hat{r}_i \in R_i^*), \quad (1)$$

where N is the number of incidents. For configurations run multiple times (the DIAGGUARD validation, Section VI), we also report $\text{pass@}k$, the fraction of incidents with $\hat{r}_i^{(j)} \in R_i^*$ for at least one of the k runs:

$$\text{pass@}k = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\exists j \in \{1, \dots, k\} : \hat{r}_i^{(j)} \in R_i^*). \quad (2)$$

d) Node and edge F1.: Let $G^* = (V^*, E^*)$ be the ground-truth service-level fault propagation graph and $G = (V, E)$ the agent’s predicted graph from its final result (Section IV-A). We compute

$$\text{Node F1} = \frac{2|V \cap V^*|}{|V| + |V^*|}, \quad (3)$$

$$\text{EdgeF1} = \frac{2|E \cap E^*|}{|E| + |E^*|}. \quad (4)$$

NodeF1 measures service-set recovery; EdgeF1, directed propagation-link reconstruction.

In Table II, rds, NodeF1, and EdgeF1 are further split by Acc@1 outcome ($\checkmark$: correct; $\times$: incorrect; avg: all cases).

e) *Causal-chain depth.*: We stratify cases by the **causal-chain depth** d^* , the shortest-path length from the root cause to the SLO-alarming service along G^* .

C. Behavior and Intent Analysis

The diagnostic goal is to localize a root cause, the set of services an agent queries and surfaces at each round indirectly reveals whether its behavior on the fault-impact surface (the services V^* and edges E^* of G^*) is advancing toward that cause, while each action’s SQL exposes the intent behind that behavior. We accordingly build two process views over the trajectories: a *behavior* view, tracking how each round navigates the fault-impact surface and uses surfaced evidence, and an *intent* view, typing each SQL action by its diagnostic intent.

1) *Impact-surface navigation and evidence utilization.*: We define three per-round labels to characterize each trajectory: a *primary service* that tracks where the investigation stands, a *navigation transition* that captures its direction, and an *evidence-utilization* label that records whether it acts on returned evidence.

a) *Per-round primary service.*: Each round touches several services, so to track this advance at round granularity we reduce it to a single *primary service*: foremost the round’s service nearest a root cause, capturing how far the round advanced. Among equally near services, it is the newly visited one, which marks a new investigation direction, or else the most frequently visited, where the round’s investigation has concentrated. Concretely, we draw candidates from the round’s own services (its *action services*, or its result services when the round issues no query) and rank them lexicographically by (i) distance from the nearest root cause along G^* , with off-path services taking a large penalty; (ii) unvisited before visited; then (iii) higher cumulative visit count (both relative to earlier rounds). Key (i) sets the round’s status (*on-path* if any candidate is on-path, i.e., the round’s own queries reach the propagation path, and *off-path* otherwise), and keys (ii) and (iii) break the remaining ties in either case.

b) *Navigation transitions.*: For each round pair $(i, i+1)$ we label a *navigation transition* from the on-path status and distance-to-root of their two primary services, capturing whether the investigation advances toward, stays level with, or moves away from a root cause (full label set in Fig. 3).

c) *Evidence utilization.*: Independently, an *evidence-utilization* label compares round $i+1$ ’s action services A_{i+1} against the on-path subset of round i ’s result services R_i , capturing whether the next round acts on the decisive evidence the previous one surfaced (Fig. 3).

2) *Intent taxonomy and Sequence patterns.*: For the intent view, every SQL action receives one intent label and a multi-valued set over six deterministic tag axes (Table I; Fig. 4).

A fixed LLM classifier (claude-opus-4.8), following the LLM-as-judge paradigm [58]–[60], labels 92,501 SQL actions by reading the compacted trajectory while anchoring on the SQL operation and returned evidence rather than the agent’s narrative. The six tag axes are derived deterministically from the SQL text; agreement and classifier accuracy are reported in Section IV-E.

We summarize the intent view in two ways: the *intent distribution*, giving each experiment’s frequency over the intents, and *intent n-grams*, capturing contiguous subsequences that surface recurring motifs and repeated-intent loops.

D. Failure-Mode Coding and Mining

We code each failed trajectory against the expected evidence for its fault type, yielding three failure families that differ in how the agent relates to decisive evidence (Table III).

1) *Coding failures against expected evidence.*: A correct diagnosis must recover sufficient evidence for at least one anomalous path from the observed symptom to a root cause on the fault-impact surface. Using the RQ2 intent and tag vocabulary, we specify each fault’s *expected evidence*, the typed queries that would confirm the relevant surface services and dependencies, and read a failed trajectory as a shortfall against it: evidence the agent omitted or misread, or reasoning not tied to that evidence.

For each failed trajectory we compare expected evidence against the agent’s actual queries and reasoning, drawing on its compacted trajectory, the ground-truth causal context, and raw telemetry when needed.

We code deviations into three families (Table III): *protocol omission* (OMIT), evidence reachable in telemetry but never queried; *semantic misread* (MIS), retrieved evidence misinterpreted or scoped to the wrong dependency; and *general reasoning* (GEN), a reasoning failure not tied to a single evidence unit.

Two authors jointly develop the codebook through open coding of failed trajectories, letting the recurring failure codes emerge and refining them until the codebook stabilizes; an LLM (claude-opus-4.8) assists evidence retrieval [51], [58], but every code is verified by hand against the cited SQL, returned evidence, and the TrainTicket service-call graph [57]. Using the fixed codebook, the two authors independently code all 154 failed trajectories (50 Sonnet, 104 Qwen); agreement is reported in Section IV-E.

E. Annotation Reliability

Three artifacts in our pipeline rest on manual judgment, and we validate each by independent double-annotation, resolving disagreements by discussion to consensus. For the ground-truth propagation graphs, one author annotated all 500 cases; a second author independently annotated a random sample of 100, matching the first author’s annotations on 93% of them. For the SRE intent labels, they judged a stratified random sample of 1,000 classifier labels, agreeing on 99% (Cohen’s $\kappa = 0.83$); against this consensus the fixed LLM classifier was correct on 97%. For the failure-mode codes, they independently

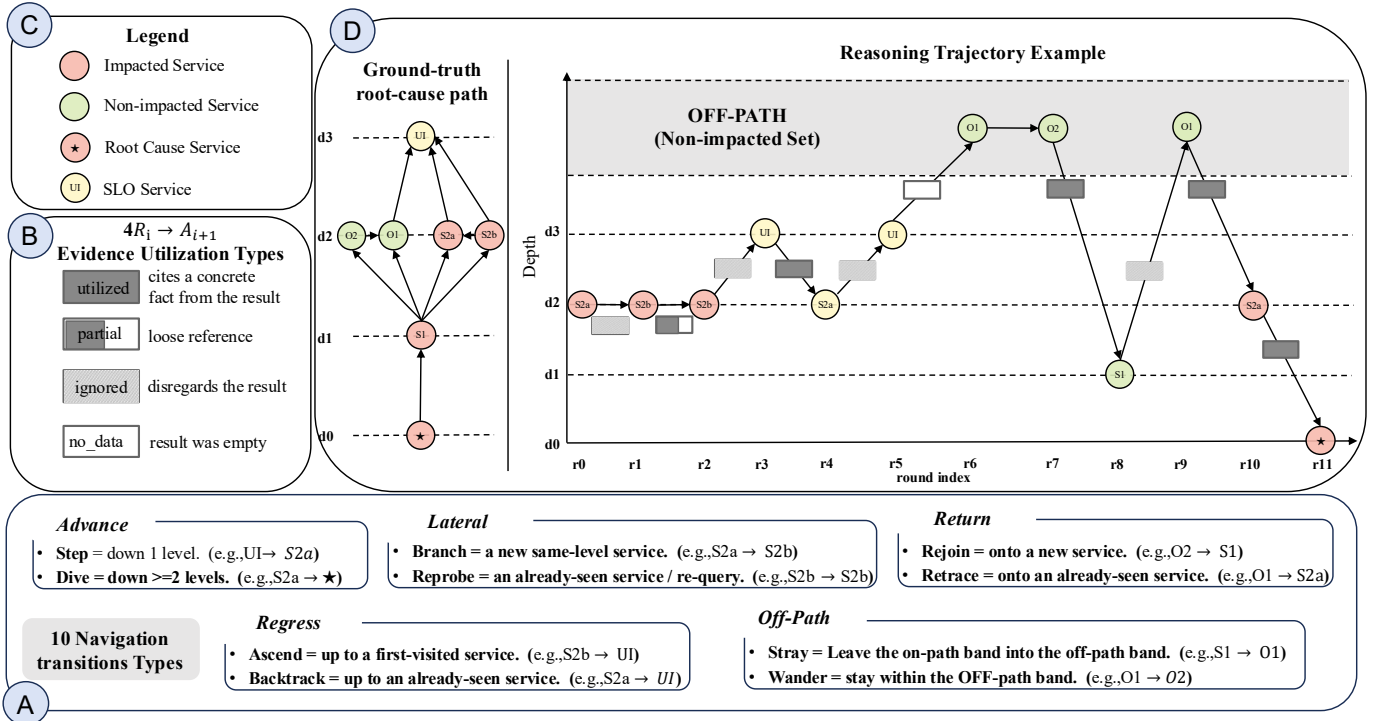


Fig. 3: Trajectory label vocabularies (RQ2 behavior view). Each reasoning round is reduced to a *primary service* at some depth on the graph (Section IV-C1). *Panel A*: the 10 navigation-transition types, grouped into five families by how the investigation moves relative to a root cause. *Panel B*: the 4 evidence-utilization types, i.e., whether a round acts on the previous round’s on-path evidence. *Panel C*: node legend for panel D. *Panel D*: a worked example plotting each round’s primary-service depth (y) over round index (x); the grey band is the off-path region and the shaded box under each round marks its evidence-utilization type. These labels are defined in Section IV-C1 and quantified in Fig. 5a.

TABLE I: The intent/tag axis: eleven SRE diagnostic intents and six SQL-derived tag axes.

Axis	Family	Name	Definition
Intent	Symptom	symptom_scan	Compares service-level symptoms, including errors, latency, status codes, payload shape, and request volume.
		log_inspect	Inspects log records for errors, exceptions, stack traces, abnormal volume, and other textual symptoms.
		keyword_search	Searches telemetry text for service names, error patterns, fault keywords, and suspicious literals.
	Topology	trace_follow	Follows trace IDs, spans, and parent-child links to localize propagation along service calls.
		callgraph	Queries caller-callee relations, span kinds, and call ratios to recover dependency direction and structure.
Tag	Resource	network_flow	Inspects request flow, failures, connectivity, packet/drop signals, and service-to-service traffic.
		resource_probe	Tests resource hypotheses with metrics for named services, pods, nodes, JVMs, DBs, or runtimes.
	Liveness	metric_dump	Retrieves broad metric values or snapshots to scan resource state before narrowing a hypothesis.
		lifecycle_probe	Checks lifecycle/emission signals such as restarts, silence, span export, pod state, and JVM state.
	Inventory	metric_enum	Enumerates metric names, schemas, coverage, resource attributes, and measurement inventory.
		service_enum	Enumerates services, pods, nodes, tables, topology entities, and telemetry inventory.
Tag	Derived	granularity	Target level over which evidence is grouped, e.g., global, node, pod, service, endpoint, or trace.
		window	Telemetry window compared by the query, e.g., abnormal, normal, or normal-abnormal contrast.
		time	Temporal shape requested by the query, e.g., snapshot, timeline, or onset analysis.
		emission	Whether the query references telemetry emission, silence, exporter, queue, or liveness fields.
		resource_family	Resource axis referenced by the query, e.g., CPU, memory, GC, pool, thread, or disk.
Tag	Derived	symptom_signal	Symptom signal referenced by the query, e.g., error, status code, latency, payload, or volume.

coded all 154 failed trajectories under the fixed codebook and assigned identical code sets on 144 (93.5%).

V. CHARACTERIZATION RESULTS

This section characterizes RCA agents along three deepening levels: RQ1 establishes outcome and cost, RQ2 explains the outcome gaps through behavior and intent, and RQ3 distills

wrong diagnoses into recurring failure modes, which Section VI later turns into an intervention.

A. RQ1: Effectiveness and Cost

a) *Framework and Model Comparison*: Table II compares agents along two axes. Across frameworks, effectiveness is set by investigative architecture: the agents that run an open-

TABLE II: RQ1 effectiveness, reasoning budget, and depth-stratified Acc@1. Across all rows, EdgeF1 trails NodeF1 by a wide margin, and Acc@1 falls as the causal chain deepens, a decline the Sonnet row largely escapes. Metrics and the $\checkmark/\times$ outcome split defined in Section IV-B; Acc@1 and depth columns in %, F1 in $[0, 1]$.

Backbone model	Framework	Acc@1	Node F1			Edge F1			Rounds (rds)			Budget / case		Acc@1 by depth d^*			
			✓	×	avg	✓	×	avg	✓	×	avg	tok	\$	$d^*=2$	$d^*=3$	$d^*=4$	$d^*=5$
Qwen	MABC	42.6	0.49	0.35	0.41	0.15	0.05	0.09	–	–	–	163	0.04	53.3	36.4	40.4	42.9
	OPENRCA	46.6	0.57	0.48	0.52	0.36	0.16	0.25	6.1	6.7	6.4	364	0.05	58.6	46.9	30.9	19.0
	TASKWEAVER	65.2	0.78	0.55	0.70	0.58	0.26	0.47	13.2	14.5	13.7	672	0.09	79.6	68.9	39.4	33.3
	AIQ	77.6	0.81	0.63	0.77	0.61	0.35	0.55	50.0	55.1	51.1	1516	0.19	86.8	82.0	59.6	42.9
	CLAUDECODE	79.6	0.81	0.58	0.76	0.65	0.28	0.57	51.3	62.7	53.6	1512	0.18	89.5	79.8	68.1	57.1
	THINKDEPTH.AI	79.2	0.83	0.59	0.78	0.66	0.33	0.59	43.1	50.7	44.7	1744	0.21	85.5	82.0	66.0	57.1
Sonnet	THINKDEPTH.AI	90.0	0.82	0.56	0.79	0.67	0.37	0.64	30.5	38.2	31.2	1465	4.64	88.2	88.6	96.8	85.7

```
SELECT time, metric, service_name, attr_k8s_pod_name, value
FROM abnormal_metrics_sum
WHERE service_name = 'ts-order-service'
AND metric IN ('jvm.memory.used', 'jvm.memory.limit',
               'jvm.cpu.time', 'jvm.thread.count',
               'db.client.connections.usage', 'queueSize')
ORDER BY time ASC, metric

intent = resource_probe
window = abnormal · time = timeline · granularity = {pod, service}
resource_family = {cpu, memory, pool, thread}
emission = true · symptom_signal = ∅
```

Fig. 4: *Intent/tag labeling of one SQL action.* A real abnormal-window metric query labeled with a single SRE *intent* (resource_probe) plus the six deterministic *tag* axes of Table I. Color-matched highlights link each SQL fragment to the tag axis it sets (Section IV-C2).

ended, adaptive investigation, requering telemetry as evidence emerges (THINKDEPTH.AI 79.2%, AIQ 77.6%, CLAUDECODE 79.6%), hold the top band in a narrow range, while the more constrained designs trail in order: TASKWEAVER (65.2%) follows a largely fixed plan, OPENRCA (46.6%) commits as soon as it can emit an answer, and MABC (42.6%) does not iterate over evidence at all. Notably, none of these top agents was built for RCA: they are open-ended agents repurposed for it, yet prove highly effective on the task.

Across backbone models, the Sonnet THINKDEPTH.AI arm reaches 90.0% Acc@1, a 10.8-pp gain over Qwen. The top-band frameworks span only 2.0pp under the same model (77.6%–79.6%), while the model swap yields over five times that gap, suggesting the backbone model as the dominant lever.

Finding 1

Adaptive, minimally constrained frameworks outperform more tightly orchestrated designs; among the top-band frameworks, the backbone model is the dominant lever.

b) Causal-Chain Depth and Reconstruction Granularity:

Causal-chain depth serves as a proxy for diagnostic difficulty: a longer chain demands more investigative steps and more evidence to reach the root cause. Empirically, accuracy declines accordingly. Every Qwen framework loses ground from depth 2 to depth 5 (CLAUDECODE 89.5 $\rightarrow$ 57.1%, AIQ 86.8 $\rightarrow$ 42.9%); on the matched THINKDEPTH.AI pair, Qwen falls

likewise (85.5 $\rightarrow$ 57.1%) while Sonnet holds flat (85.7–96.8%), widening the backbone-model gap on the hardest cases.

Acc@1 and surface reconstruction are positively coupled across Table II, yet the backbone-model swap lifts localization without lifting the reconstruction ceiling. Weaker agents do not merely select the wrong final service: they reconstruct less of the fault-impact surface even on their correctly localized cases ($\checkmark$ NodeF1 rises from 0.49 for MABC to 0.83 for THINKDEPTH.AI, $\checkmark$ EdgeF1 from 0.15 to 0.66). Even correct localizations, however, rest on partial causal maps: $\checkmark$ EdgeF1 never exceeds 0.67, and the model swap that raises Acc@1 by 10.8pp leaves this ceiling nearly untouched (NodeF1 0.82 vs. 0.83, EdgeF1 0.67 vs. 0.66). In every row, EdgeF1 falls well below NodeF1 (MABC: 0.41 vs. 0.09; CLAUDECODE: 0.76 vs. 0.57), and failure widens this imbalance: from correct to incorrect runs, NodeF1 drops by 16–32% while EdgeF1 drops by 43–67%, so failed runs still detect most anomalous services yet no longer chain them into a propagation path. Identifying anomalous nodes requires only detecting aberrant telemetry; recovering directed edges requires inferring how the fault propagates between services, making edge reconstruction the harder and more discriminative capability.

Finding 2

Accuracy degrades with causal-chain depth, a decline the stronger backbone model largely escapes. Reconstructing the fault-impact surface is harder than selecting the correct final answer and remains partial even when localization succeeds; within the reconstruction, recovering directed propagation edges proves more demanding than identifying anomalous nodes.

c) Reasoning Budget and Diagnostic Outcome:

Budget scales with investigative openness: the top-band agents sustain 45–54 effective rounds, over 1500 ktok, and roughly \$0.2 per case, whereas the constrained agents stop far sooner (under 700 ktok, at most \$0.09). Within THINKDEPTH.AI, Sonnet stops in fewer rounds than Qwen, locking onto decisive evidence quickly where Qwen enumerates more broadly, though Sonnet’s higher per-token price still dominates per-case cost.

Yet one pattern is shared across all agents: in every iterative arm, incorrect cases run more rounds than correct ones, by up

to 11.4 rounds. The extra rounds buy no partial credit either: incorrect runs trail correct ones on both F1 metrics, so the added budget reflects unproductive search rather than evidence accumulation.

Finding 3

Reasoning budget varies across frameworks and backbone models but does not predict diagnostic quality: failed runs consume more rounds than successful ones, as unproductive search inflates cost without improving outcomes.

B. RQ2: Agent Behavior

Having established outcome and cost differences, we now examine the diagnostic process behind them. Because RQ2–RQ3 analyze SQL-call trajectories, they exclude MABC and OPENRCA, which expose no comparable traces. We center on THINKDEPTH.AI’s Qwen and Sonnet arms, the only matched pair; the Qwen arm is representative because all Qwen SQL-call frameworks show similar behavior and intent patterns.

a) *Surface Navigation and Evidence Grounding*: Correct runs ground each round in retrieved evidence; incorrect runs drift off-path or ignore decisive clues (Fig. 5a). Specifically, correct runs lean on Reprobe, re-querying on-path services at the same depth (higher than incorrect runs in both arms), whereas incorrect runs oscillate on and off the surface (higher Stray/Wander and Retrace). The two arms navigate in similar proportions.

Evidence grounding, not navigation volume, is where the two backbone models diverge. Correct runs act on the evidence they retrieve (higher utilized), whereas incorrect runs more often ignore results or hit empty queries (higher ignored and no_data). Sonnet acts on the prior result (utilized or partial) on 54.3% of rounds versus Qwen’s 24.9%. In effect, Sonnet’s backtracks and advances are evidence-guided confirmations, while Qwen swings between shallow and deep nodes regardless of the returned evidence.

Finding 4

Successful RCA stay on the fault-impact surface and ground each round in retrieved evidence; incorrect RCA drift off the surface or ignore decisive clues.

b) *Intent Repertoire and Fault-Type Sensitivity*: The two backbone models deploy qualitatively different diagnostic strategies, with Sonnet pivoting mid-trajectory to resource- and topology-oriented intents where Qwen persists in shallow trace and log loops (Fig. 5b). Over the full eleven-intent profile, the Sonnet arm is the least similar to every Qwen arm (cosine similarity 0.907–0.928), while the four Qwen frameworks cluster tightly (0.947–0.989).

Both arms open with a shared symptom-scanning phase, after which their strategies diverge. Qwen stays trace- and log-bound: as the trajectory advances it relies increasingly on fuzzy keyword_search over log and trace text and on trace_follow span enumeration, and although it inventories metrics via metric_enum and metric_dump, it rarely

drills into them. Sonnet runs the same early symptom/log/trace survey but then pivots in mid-trajectory to exploring the metric query space (metric_enum), recovering dependency structure with callgraph, and drilling named resources and flows via resource_probe (used in 82.8% of its trajectories vs. 16.6% for Qwen) and network_flow. This maps onto per-fault accuracy: Qwen trails Sonnet most on categories requiring named-resource metrics (RESOURCE 75% vs. 94%, CODE (JVM) 66% vs. 93%), while NETWORK nearly closes (82% vs. 89%) and HTTP is tied (88% vs. 87%).

Qwen further perseverates in shallow loops that Sonnet avoids: intent 5-grams such as sustained trace-following (trace⁵, 0.96 per trajectory) and log_inspect/symptom_scan oscillation recur far more in the Qwen arm than in Sonnet (≤ 0.12 per trajectory), the sequence-level signature of its narrow repertoire.

Finding 5

A richer diagnostic intent repertoire corresponds to broader, SRE-style search, while a narrow one keeps the agent in shallow loops over a limited slice of the evidence, producing fault-type-specific accuracy gaps that concentrate on the types requiring deeper evidence.

C. RQ3: Failure Taxonomy

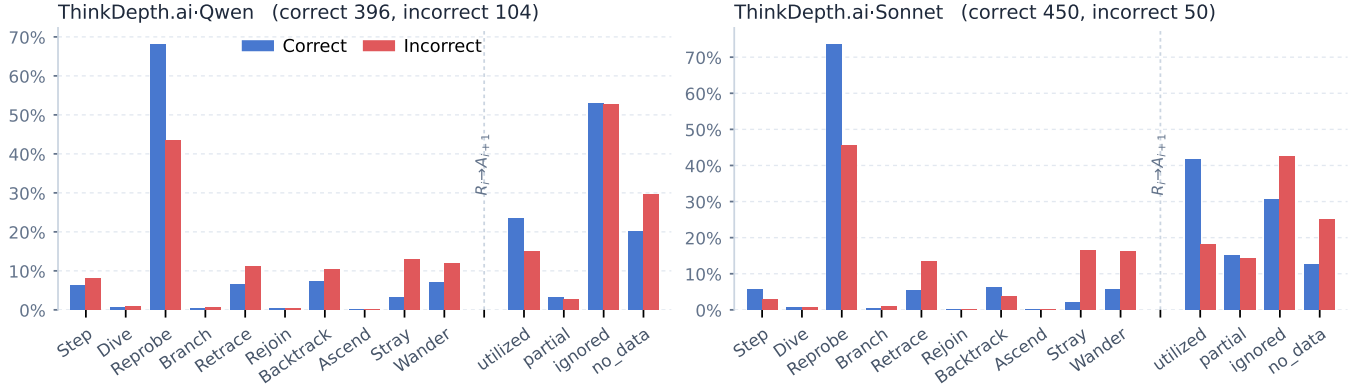
The behavioral patterns above show *how* runs diverge; we now ask *why* wrong diagnoses occur. Wrong diagnoses reduce to three evidence-handling failure families that differ in how the agent relates to decisive evidence (Table III).

a) *Failure families and cross-model patterns*: The general-reasoning failures in Table III are untied to any single signal: a model that cannot aggregate scattered evidence or drill deep enough to discard distractors locks onto an early anchor (GEN1); an overconfident one fills gaps by fabricating an out-of-schema cause (GEN2); and one without the persistence to retry a failed or empty query abandons the evidence channel altogether (GEN3).

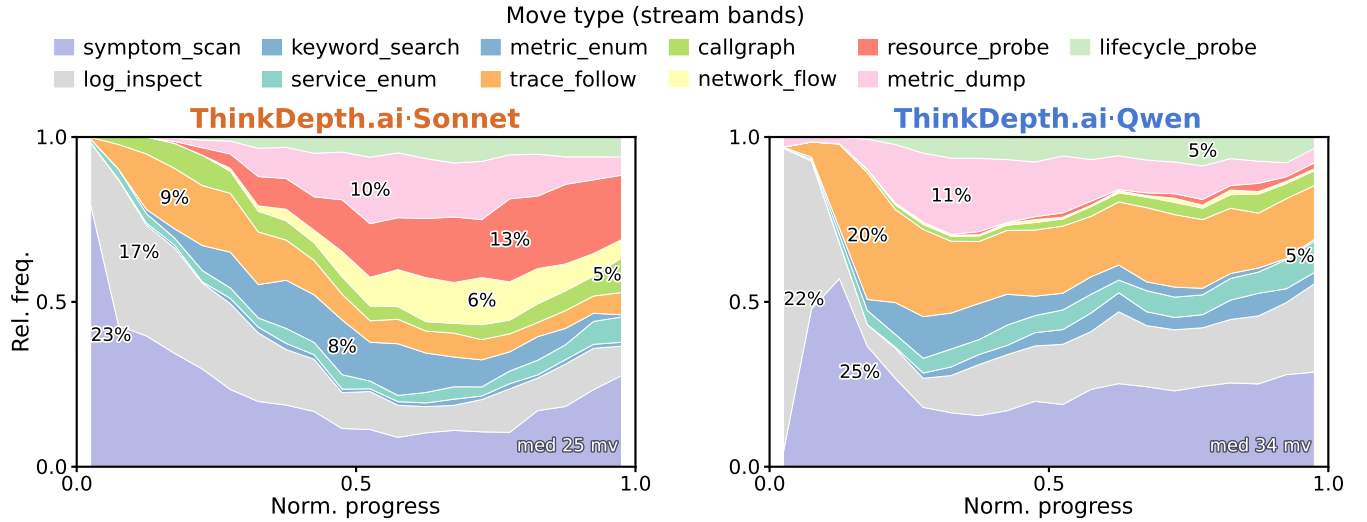
The evidence-anchored families track the two backbone models’ reasoning posture. Qwen fails predominantly by omission, Sonnet by misreading. Qwen has less working knowledge of the RCA problem domain, so it stays redundantly confined to the most superficial, easily found signals (OMIT1, OMIT3, OMIT4) and rarely redirects its exploration from the evidence already gathered (GEN1). Sonnet has a stronger ability to hypothesize new investigation directions from the evidence, but its strong associative reach sometimes errs through hallucination (GEN2), and its synthesis often falls short of pinning down the crux among contradictory signals (MIS2, MIS3).

Finding 6

Agent failures reduce to three evidence-handling modes: decisive evidence is never gathered, is gathered but misread, or is overridden by ungrounded reasoning.



(a) **Navigation and evidence-utilization label shares**, correct (blue) vs. incorrect (red) runs per arm. In the navigation block (left), the correct–incorrect contrast is consistent across both arms; in the evidence-utilization block (right, after the dashed line), the two base models diverge sharply. Labels defined in Fig. 3; bars macro-averaged per trajectory.



(b) **SRE diagnostic-intent composition** over normalized trajectory progress. Both arms share a symptom-dominated opening phase; band shapes diverge from mid-trajectory onward, with the Sonnet arm shifting weight toward intent categories largely absent in Qwen. Band thickness is each intent’s relative frequency (colors: the 11 intents of Table I); in-plot labels give overall shares.

Fig. 5: **RQ2 behavior and intent analysis**: what separates correct from incorrect runs (a) and what drives the backbone-model gap in diagnostic strategy (b).

Whether distilling these recurring modes into defenses can improve localization beyond the characterization setting remains to be tested.

VI. INTERVENTION AND VALIDATION

This section tests whether the mined failure modes can guide practical improvement beyond the characterization setting: we design DIAGGUARD from the RQ3 taxonomy (Section VI-A) and validate it on the held-out setting (Section VI-B).

A held-out incident illustrates the design (Fig. 6): a disk-I/O fault on the TiKV storage node `tldb-tikv-0` surfaces only as downstream tail latency, so localizing it requires descending past the service tier and chronic background noise to the storage node. The baseline THINKDEPTH.AI agent fails through several already-catalogued modes at once (MIS2, OMIT1, OMIT3,

OMIT5, OMIT6, GEN3; Table III), confirming that the modes recur on an unseen model and dataset. Under DIAGGUARD they are overcome, localizing the storage node as the true root cause.

A. DIAGGUARD Design

The taxonomy of Section V-C is not only a diagnosis of why agents fail: it is a design specification. Its three families expose a single structural deficit. A bare reasoning loop is at once *under-grounded*, committing to evidence it never gathered or to states it fabricated, and *under-verified*, never re-examining a reading or a conclusion before it commits. We read each mined mode as a requirement and close it with a dedicated, answer-agnostic defense, yielding **DIAGGUARD**: a failure-taxonomy-

TABLE III: Three evidence-handling failure families. Each family addresses a different relationship between the agent and decisive evidence: never gathered (OMIT), gathered but misread (MIS), or overridden by ungrounded reasoning (GEN).

Family	Mode	Definition	Consequence	Sonnet	Qwen
Protocol omissions	OMIT1 Baseline gap	No baseline-window comparison.	Pre-existing signal misjudged as incident evidence.	26.0%	51.9%
	OMIT2 Edge untested	No caller-callee edge test.	Propagation edge lacks span-level support.	54.0%	68.3%
	OMIT3 Resource gap	No resource/lifecycle probe using concrete names.	Decisive memory, restart, etc. signal remains unseen.	34.0%	63.5%
	OMIT4 Ranking gap	No peer ranking on the same signal.	Collateral spike is treated as uniquely causal.	2.0%	25.0%
	OMIT5 Granularity gap	No pod, endpoint, edge, or metric-level drill-down.	Failing pod, endpoint, edge, or metric remains unidentified.	18.0%	16.3%
	OMIT6 Timing gap	No timing check for the proposed cause.	Causal order lacks temporal corroboration.	4.0%	8.7%
Semantic misreads	MIS1 Silence-as-health	Silence, flatness, or missing data read as health.	Suppressed callee fault is cleared as healthy.	30.0%	44.2%
	MIS2 Saliency bias	Loudest signal over-trusted.	Noisy victim service is selected as the cause.	42.0%	32.7%
	MIS3 Semantics misread	Telemetry text read with wrong service semantics.	Caller-side client fault attributed to downstream callee.	86.0%	61.5%
General reasoning	GEN1 Anchor lock	Early anchor retained after conflicting evidence.	Contradictory evidence is discounted in the final answer.	28.0%	34.6%
	GEN2 Schema-out claim	Out-of-schema causal state fabricated.	Unsupported out-of-schema state is returned as the cause.	82.0%	20.2%
	GEN3 Friction stop	Evidence channel dropped after tool friction.	Resource evidence remains unavailable after tool friction.	0%	12.5%

Note. Percentages report within-model failure-pool prevalence ($n=50$ Sonnet, $n=104$ Qwen); subcategories may overlap and do not sum to 100%.

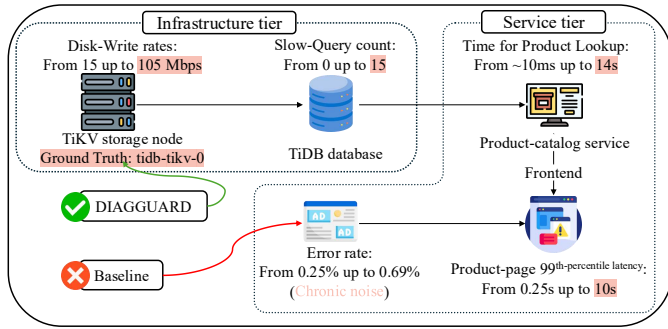


Fig. 6: **Motivating incident (Seed 2.0 Pro / AIOps 2025).** An injected disk-I/O fault on the TiKV storage node `tidb-tikv-0` surfaces only as downstream tail latency; red-highlighted values mark abnormal-vs-normal shifts. The baseline (red cross) anchors on the chronically noisy Ad service and never leaves the service tier, whereas DIAGGUARD (green check) descends through the infrastructure tier to the true root cause `tidb-tikv-0` (Section VI).

grounded, defense-in-depth diagnostic architecture that wraps a reasoning core in two complementary defenses (Fig. 7).

The core is the *Diagnostician*, the THINKDEPTH.AI agent of Section III adapted to the AIOps 2025 telemetry interface; run alone, it is the baseline of Section VI-B. Around it sit two answer-agnostic defenses that adapt techniques studied in isolation for general agents: the *Grounder* (*agentic grounding* [61]) and the *Verifier* (*inference-time verification* [62]), shown in Fig. 7. Complementary, they span the three families between them: evidence that is never gathered, gathered but misread, or overridden by ungrounded reasoning. Each check is mapped from a specific mined mode and frozen before the validation runs; the excerpt in Section VI-B, for instance, is mapped from the baseline-gap mode OMIT1, under which a chronic signal is misread as incident evidence.

B. Validation

With DIAGGUARD designed, we validate it on the held-out setting along two axes: whether it improves localization, then

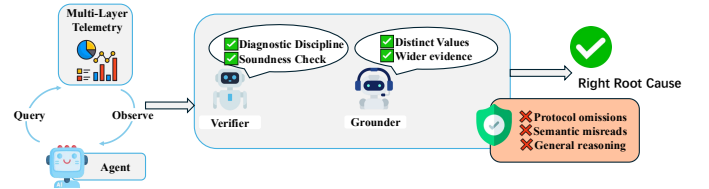


Fig. 7: **The DIAGGUARD architecture.** A reasoning core is wrapped by two answer-agnostic defenses derived from the RQ3 failure taxonomy. The *Grounder* supplies grounding, confronting the agent with the telemetry that exists (distinct values, wider evidence); the *Verifier* supplies verification, a diagnostic-discipline self-audit and a soundness check before any commitment. Together they close the three failure families (Table III) that a bare reasoning loop leaves open (Section VI-A).

how much each component contributes to that improvement.

a) *RQ4: DIAGGUARD effectiveness:* Table IV shows that DIAGGUARD raises Acc@1 from 43.5% to 52.5% (+9.0), pass@3 from 56.9% to 67.1% (+10.2), and pass@5 from 62.3% to 73.0% (+10.7). The improvement holds across every fault type, with per-type gains from +1.0 (DNS) to +20.8 (I/O). Absolute accuracy is lower than on RCABench for two reasons: half of its fault types lie below the service tier and raise no SLO, so the agent starts blind; and the target granularity varies by fault: each AIOps 2025 fault must be localized at one prescribed layer (a service, a pod, or a node), rather than at the single service level we score on RCABench.

Verifier self-audit (excerpt)

Establish a baseline before relying on any signal: compare the abnormal window to the normal window for the same entity and the same metric, log-pattern, or volume. If an error, a latency, or a ‘‘first failure’’ already exists in the normal period, it is chronic background and cannot be the root-cause anchor.

TABLE IV: DIAGGUARD versus the baseline on the held-out setting (Seed 2.0 Pro / AIOps 2025). Gains hold across all nine fault types (lower panel). Net.: network attack; Chg.: erroneous change; Cfg.: misconfiguration.

Configuration	Acc@1	pass@3	pass@5
Baseline	43.5 $\pm$ 0.6	56.9	62.3
DIAGGUARD (ours)	52.5$\pm$1.1	67.1	73.0

Per fault type (Acc@1)									
	Net.	Node	Pod	JVM	Stress	I/O	Chg.	DNS	Cfg.
Baseline	78.1	4.1	42.7	65.1	52.4	2.1	32.4	43.8	75.6
DIAGGUARD	88.2	19.5	45.0	67.3	62.4	22.9	39.0	44.8	84.4

TABLE V: Component ablation of DIAGGUARD. The Δ column shows near-additive gains that do not track the budget columns (rds, ktok, \$): the costliest arm gains least, and the full stack gains most while spending less than it. Acc@1 is mean $\pm$ sd over five runs.

Configuration	Acc@1		pass@3	pass@5	Budget / incident		
	%	Δ	%	%	rds	ktok	\$
Baseline (w/o both)	43.5 $\pm$ 0.6	—	56.9	62.3	14.1	176.8	0.09
w/o Grounder	47.6 $\pm$ 1.6	+4.1	61.3	65.8	19.2	343.8	0.17
w/o Verifier	48.0 $\pm$ 0.9	+4.5	62.0	67.3	16.3	212.1	0.10
DIAGGUARD (full)	52.5$\pm$1.1	+9.0	67.1	73.0	18.7	318.5	0.16

Finding 7

Diagnostic experience distilled from the mined failure modes remains effective on a held-out backbone model, dataset, and service topology.

b) RQ5: Component contribution: Table V ablates the two defenses. Each helps on its own by a comparable margin: the Grounder alone (*w/o Verifier*) raises Acc@1 by +4.5 and the Verifier alone (*w/o Grounder*) by +4.1, with the full DIAGGUARD reaching +9.0 and the same ordering at pass@3 and pass@5. We thus read the Grounder’s breadth and the Verifier’s discipline as complementary and non-redundant, with each contributing a separable gain.

VII. THREATS TO VALIDITY

External validity. Our characterization (RQ1–RQ3) rests on a single topology (RCABench’s TrainTicket), the fixed sample, and the chosen agents, and may not transfer to other topologies, stacks, or deployments; DIAGGUARD’s transfer to a held-out backbone model, dataset, and service topology partly mitigates this.

Internal validity. Although our comparisons vary one factor at a time, residual implementation differences across frameworks and models may still confound them.

Construct validity. Our process labels rest on manual annotation, LLM-assisted classification, and human review; fixed protocols, double coding, and released artifacts reduce but cannot eliminate annotation subjectivity and judge bias.

Conclusion validity. Each characterization configuration (RQ1–RQ3) is run once without significance testing, so its

reported differences are indicative; the validation (RQ4–RQ5) reports pass@k and mean $\pm$ sd over five runs.

VIII. CONCLUSION

We presented a process-level study of how LLM agents reach a root-cause diagnosis, not merely whether it is correct. Against manually annotated fault propagation paths, we characterized agent reasoning trajectories across backbone models along outcome, behavior, and intent. We find that final-answer accuracy masks how completely an agent reconstructs the fault-impact surface: recovering the propagation path is harder than naming the root-cause service, and even correctly localized runs leave the path incomplete. Effective diagnosis is an evidence-grounded, on-surface process whose intent breadth sets its search depth; failing runs drift off the surface or leave decisive evidence unused. Wrong diagnoses reduce to a small taxonomy of omitted or misread evidence and ungrounded reasoning, and distilling it into DIAGGUARD improves localization on a held-out backbone model, dataset, and service topology. Trajectory-level analysis thus exposes agent capability boundaries that final-answer evaluation alone cannot reveal. Future work will turn trajectory labels into reinforcement-learning signals and target harder, more production-like incidents.

REFERENCES

- [1] X. Guo, X. Peng, H. Wang, W. Li, H. Jiang, D. Ding, T. Xie, and L. Su, “Graph-based trace analysis for microservice architecture understanding and problem diagnosis,” in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. New York, NY: ACM, 2020, pp. 1387–1397.
- [2] Google SRE Team, “Effective troubleshooting,” <https://sre.google/sre-book/effective-troubleshooting/>, 2016, accessed: 2024-12-01.
- [3] N. Fu, G. Cheng, Y. Teng, G. Dai, S. Yu, and Z. Chen, “Intelligent root cause localization in microservice systems: A survey and new perspectives,” *ACM Computing Surveys*, vol. 57, no. 12, 2025.
- [4] S. Zhang, S. Xia, W. Fan, B. Shi, X. Xiong, Z. Zhong, M. Ma, Y. Sun, and D. Pei, “Failure diagnosis in microservice systems: A comprehensive survey and analysis,” *ACM Transactions on Software Engineering and Methodology*, 2024.
- [5] A. Fang, H. Yang, H. Dong, Q. Lu, J. Xu, and P. He, “A goal-driven survey on root cause analysis,” *arXiv preprint arXiv:2510.19593*, 2025.
- [6] Z. Li, J. Yu, Z. Huang, and Y. Huang, “Surveying root cause analysis techniques: A comprehensive review of aspects for multi-service applications,” *IEEE Transactions on Services Computing*, vol. 19, no. 1, pp. 808–825, 2026.
- [7] I. Bouzenia and M. Pradel, “Understanding software engineering agents: A study of thought-action-result trajectories,” *arXiv preprint arXiv:2506.18824*, 2025.
- [8] M. Du, B. Xu, C. Zhu, X. Wang, and Z. Mao, “Deepresearch bench: A comprehensive benchmark for deep research agents,” *arXiv preprint arXiv:2506.11763*, 2025.
- [9] Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gao, L. Shi, Y. Cao, X. Gao, H. Fan, M. Wen *et al.*, “Automatic root cause analysis via large language models for cloud incidents,” in *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024, pp. 674–688.
- [10] Y. Gao, “An framework for fault localization in microservices,” *Available at SSRN 5420754*, 2025.
- [11] W. Zhang, H. Guo, J. Yang, Z. Tian, Y. Zhang, Y. Chaoran, Z. Li, T. Li, X. Shi, L. Zheng *et al.*, “mabc: multi-agent blockchain-inspired collaboration for root cause analysis in micro-services architecture,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 4017–4033.
- [12] J. Xu, Q. Zhang, Z. Zhong, S. He, C. Zhang, Q. Lin, D. Pei, P. He, D. Zhang, and Q. Zhang, “Openrca: Can large language models locate the root cause of software failures?” in *The thirteenth international conference on learning representations*, 2025.

- [13] A. Fang, S. Zhang, Y. Yang, H. Wu, J. Xu, X. Wang, R. Wang, M. Wang, Q. Lu, and P. He, "Rethinking the evaluation of microservice rca with a fault propagation-aware benchmark," *arXiv preprint arXiv:2510.04711*, 2025.
- [14] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, D. Liu, Q. Xiang, and C. He, "Latent error prediction and fault localization for microservice applications by learning from system trace logs," in *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2019, pp. 683–694.
- [15] H. Shan, Y. Chen, H. Liu, Y. Zhang, X. Xiao, X. He, M. Li, and W. Ding, "e-diagnosis: Unsupervised and real-time diagnosis of small-window long-tail latency in large-scale microservice platforms," in *The World Wide Web Conference, WWW 2019, San Francisco, CA, USA, May 13–17, 2019*, L. Liu, R. W. White, A. Mantrach, F. Silvestri, J. J. McAuley, R. Baeza-Yates, and L. Zia, Eds. ACM, 2019, pp. 3215–3222. [Online]. Available: <https://doi.org/10.1145/3308558.3313653>
- [16] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "MicroRCA: Root cause localization of performance issues in microservices," in *NOMS 2020 – 2020 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2020.
- [17] G. Yu, P. Chen, H. Chen, Z. Guan, Z. Huang, L. Jing, T. Weng, X. Sun, and X. Li, "Microrank: End-to-end latency issue localization with extended spectrum analysis in microservice environments," in *WWW '21: The Web Conference 2021, Virtual Event / Ljubljana, Slovenia, April 19–23, 2021*, J. Leskovec, M. Grobelnik, M. Najork, J. Tang, and L. Zia, Eds. ACM / IW3C2, 2021, pp. 3087–3098. [Online]. Available: <https://doi.org/10.1145/3442381.3449905>
- [18] S. Chakraborty, S. Garg, S. Agarwal, A. Chauhan, and S. K. Saini, "CausIL: Causal graph for instance level microservice data," in *Proceedings of the ACM Web Conference 2023 (WWW)*. ACM, 2023, pp. 2905–2915.
- [19] M. Li, Z. Li, K. Yin, X. Nie, W. Zhang, K. Sui, and D. Pei, "Causal inference-based root cause analysis for online service systems with intervention recognition," in *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14–18, 2022*, A. Zhang and H. Rangwala, Eds. ACM, 2022, pp. 3230–3240. [Online]. Available: <https://doi.org/10.1145/3534678.3539041>
- [20] L. Pham, H. Ha, X. Zhang, and H. Zhang, "TORAI: Multi-source root cause analysis for blind spots in microservice service call graph," in *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*, 2026.
- [21] Y. Li, Y. Wu, J. Liu, Z. Jiang, Z. Chen, G. Yu, and M. R. Lyu, "COCA: generative root cause analysis for distributed systems with code knowledge," in *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 2025, pp. 1346–1358.
- [22] S. Zhang, A. Fang, Y. Yang, R. Cheng, X. Tang, and P. He, "Dynacausal: Dynamic causality-aware root cause analysis for distributed microservices," *arXiv preprint arXiv:2510.22613*, 2025.
- [23] C. Lee, T. Yang, Z. Chen, Y. Su, and M. R. Lyu, "Eadro: An end-to-end troubleshooting framework for microservices on multi-source data," in *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 1750–1762.
- [24] G. Yu, P. Chen, Y. Li, H. Chen, X. Li, and Z. Zheng, "Nezha: Interpretable fine-grained root cause analysis for microservices on multi-modal observability data," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023, pp. 553–565.
- [25] L. Zheng, Z. Chen, J. He, and H. Chen, "MULAN: Multi-modal causal structure learning and root cause analysis for microservice systems," in *Proceedings of the ACM on Web Conference 2024 (WWW)*, 2024, pp. 4107–4116.
- [26] Anthropic, "Building effective agents," <https://www.anthropic.com/research/building-effective-agents>, 2024, accessed: 2026.
- [27] D. Zhang, X. Zhang, C. Bansal, P. Las-Casas, R. Fonseca, and S. Rajmohan, "Lm-pace: Confidence estimation by large language models for effective root causing of cloud incidents," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 388–398.
- [28] Y. Han, Q. Du, Y. Huang, J. Wu, F. Tian, and C. He, "The potential of one-shot failure root cause analysis: Collaboration of the large language model and small classifier," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 931–943.
- [29] X. Zhang, S. Ghosh, C. Bansal, R. Wang, M. Ma, Y. Kang, and S. Rajmohan, "Automated root causing of cloud incidents using in-context learning with gpt-4," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 266–277.
- [30] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan, "Recommending root-cause and mitigation steps for cloud incidents using large language models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 1737–1749.
- [31] D. Goel, F. Husain, A. Singh, S. Ghosh, A. Parayil, C. Bansal, X. Zhang, and S. Rajmohan, "X-lifecycle learning for cloud incident management using llms," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 417–428.
- [32] A. Hrusto, P. Runeson, and M. C. Ohlsson, "Autonomous monitors for detecting failures early and reporting interpretable alerts in cloud operations," in *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*, 2024, pp. 47–57.
- [33] Y. Sun, T. Zheng, X. Wen, W. Kuang, H. Liu, S. Zhang, C. Shen, B. Wu, and D. Pei, "A multimodal intelligent change assessment framework for microservice systems based on large language models," in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, 2025, pp. 378–388.
- [34] S. Liang, P. Chen, B. Tian, G. Tan, M. Xu, Y. Qu, Y. Zhao, Y. Shang, and C. Tan, "MetaRCA: A generalizable root cause analysis framework for cloud-native systems powered by meta causal knowledge," in *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*, 2026.
- [35] C. Pei, Z. Wang, F. Liu, Z. Li, Y. Liu, X. He, R. Kang, T. Zhang, J. Chen, J. Li *et al.*, "Flow-of-action: Sop enhanced llm-based multi-agent system for root cause analysis," in *Companion Proceedings of the ACM on Web Conference 2025*, 2025, pp. 422–431.
- [36] D. Roy, X. Zhang, R. Bhav, C. Bansal, P. Las-Casas, R. Fonseca, and S. Rajmohan, "Exploring llm-based agents for root cause analysis," in *Companion proceedings of the 32nd ACM international conference on the foundations of software engineering*, 2024, pp. 208–219.
- [37] Z. Wang, Z. Liu, Y. Zhang, A. Zhong, J. Wang, F. Yin, L. Fan, L. Wu, and Q. Wen, "RCAgent: Cloud root cause analysis by autonomous agents with tool-augmented large language models," in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, CIKM 2024, Boise, ID, USA, October 21–25, 2024*, E. Serra and F. Spezzano, Eds. ACM, 2024, pp. 4966–4974.
- [38] L. Zhang, T. Jia, Y. Zhai, L. Pan, C. Duan, M. He, M. Jia, and Y. Li, "Agentic memory enhanced recursive reasoning for root cause localization in microservices," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '26)*. ACM, 2026.
- [39] Y. Chen, J. Pan, J. Clark, Y. Su, N. Zheutlin, B. Bhavya, R. R. Arora, Y. Deng, S. Jha, and T. Xu, "STRATUS: A multi-agent system for autonomous reliability engineering of modern clouds," in *Advances in Neural Information Processing Systems*, 2025. [Online]. Available: <https://neurips.cc/virtual/2025/poster/116834>
- [40] L. Zhang, Y. Zhai, T. Jia, C. Duan, S. Yu, J. Gao, B. Ding, Z. Wu, and Y. Li, "Thinkfl: Self-refining failure localization for microservice systems via reinforcement fine-tuning," *ACM Transactions on Software Engineering and Methodology*, 2026, published online January 19, 2026 (Just Accepted).
- [41] Z. Xie, Z. Li, X. He, S. Zhang, L. Xu, Y. Yang, T. Zhang, J. Chen, R. Shi, and D. Pei, "Foundroot: Towards foundation model for root cause analysis via structured deep thinking," in *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering*, ser. ICSE '26. ACM, 2026.
- [42] R. Li, M. Du, B. Xu, C. Zhu, X. Wang, and Z. Mao, "Deepresearch bench ii: Diagnosing deep research agents via rubrics from expert report," *arXiv preprint arXiv:2601.08536*, 2026.
- [43] NVIDIA AI Blueprints, "AIQ research assistant," <https://github.com/NVIDIA-AI-Blueprints/aiq/tree/drbl>, 2025.
- [44] B. Qiao, L. Li, X. Zhang, S. He, Y. Kang, C. Zhang, F. Yang, H. Dong, J. Zhang, L. Wang *et al.*, "Taskweaver: A code-first agent framework," *arXiv preprint arXiv:2311.17541*, 2023.

- [45] ThinkDepth.ai, “Deep Research,” https://github.com/thinkdepthai/Deep_Research, 2025.
- [46] J. Lu, B. Pan, J. Chen, Y. Feng, J. Hu, Y. Peng, and W. Chen, “Agentlens: Visual analysis for agent behaviors in llm-based autonomous systems,” *IEEE Transactions on Visualization and Computer Graphics*, 2024.
- [47] W. Epperson, G. Bansal, V. C. Dibia, A. Fourny, J. Gerrits, E. Zhu, and S. Amershi, “Interactive debugging and steering of multi-agent ai systems,” in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025, pp. 1–15.
- [48] P. Gupta, S. Kirtania, A. Singha, S. Gulwani, A. Radhakrishna, G. Soares, and S. Shi, “Metareflection: Learning instructions for language agents using past reflections,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 8369–8385.
- [49] Y. Song, D. Yin, X. Yue, J. Huang, S. Li, and B. Y. Lin, “Trial and error: Exploration-based trajectory optimization of llm agents,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 7584–7600.
- [50] Z. Deng, Z. Dou, Y. Zhu, J.-R. Wen, R. Xiong, M. Wang, and W. Chen, “From novice to expert: Llm agent policy optimization via step-wise reinforcement learning,” *arXiv preprint arXiv:2411.03817*, 2024.
- [51] S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen, and Q. Wu, “Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems,” in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, spotlight.
- [52] E. Riddell, J. Riddell, G. Sun, M. Antkiewicz, and K. Czarnecki, “Stalled, biased, and confused: Uncovering reasoning failures in LLMs for cloud-based root cause analysis,” in *Proceedings of the 3rd ACM International Conference on AI Foundation Models and Software Engineering (FORGE)*, 2026.
- [53] T. Kim, W. Park, H. Yun, and K. Lee, “Why do AI agents systematically fail at cloud root cause analysis?” 2026.
- [54] J. Wang, X. Liao, and W. Wu, “STAR: A stage-attributed triage and repair framework for RCA agents in microservices,” 2026.
- [55] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *The Eleventh International Conference on Learning Representations*, 2023.
- [56] AIOps Challenge Committee, “AIOps Challenge 2025: Live Benchmark for Automated Root Cause Analysis,” <https://www.aiops.cn/gitlab/aiops-live-benchmark/aiopschallenge2025>, 2025.
- [57] FudanSELab, “Train Ticket: A Benchmark Microservice System,” <https://github.com/FudanSELab/train-ticket>, 2026, accessed: 2026-02-26.
- [58] M. Zhuge, C. Zhao, D. R. Ashley, W. Wang, D. Khizbullin, Y. Xiong, Z. Liu, E. Chang, R. Krishnamoorthi, Y. Tian, Y. Shi, V. Chandra, and J. Schmidhuber, “Agent-as-a-judge: Evaluate agents with agents,” in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 80569–80611.
- [59] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, Y. Wang, and J. Guo, “A survey on LLM-as-a-judge,” 2024.
- [60] H. Li, Q. Dong, J. Chen, H. Su, Y. Zhou, Q. Ai, Z. Ye, and Y. Liu, “LLMs-as-Judges: A comprehensive survey on LLM-based evaluation methods,” 2024.
- [61] M. Deng, A. Ramachandran, C. Xu, L. Hu, Z. Yao, A. Datta, and H. Zhang, “ReFoRCE: A text-to-SQL agent with self-refinement, format restriction, and column exploration,” 2025.
- [62] V. Venkatesh, M. Rathee, and A. Anand, “Trust but verify! a survey on verification design for test-time scaling,” 2025.